LANGUAGE AGNOSTIC AI CHATBOT

A Project Report
submitted
in partial fulfilment of the requirements for the degree of

Bachelor of Technology (B.Tech.)

in

Information Technology

(Session: 2026-27)

by

ADHEEP OJHA	(Roll No. - 2405250139001)
MONIKA SINGH	(Roll No. - 2305250130034)
KUMARI KIRAN	(Roll No. - 2305250130027)
ADITI YADAV	(Roll No. - 2305251540003)

Under the supervision of
Miss Purva Rai
(Assistant Professor)

Buddha Institute Of Technology GIDA, Gorakhpur
Affiliated to
Dr. A.P.J. Abdul Kalam Technical University, Lucknow
Uttar Pradesh, India

October, 2026

DECLARATION

We hereby declare that the work presented in this report entitled "**Language Agnostic AI Chatbot**" was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute.

We have given due credit to the original authors/sources for all the words, ideas, diagrams, graphics, computer programs, experiments and results that are not our original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors/sources.

We affirm that no portion of our work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, we shall be fully responsible and answerable.

ADHEEP OJHA	Roll No. - 2405250139001	_____
MONIKA SINGH	Roll No. - 2305250130034	_____
KUMARI KIRAN	Roll No. - 2305250130027	_____
ADITI YADAV	Roll No. - 2305251540003	_____

(Candidates' Signature)

Date: _____

Department of Information Technology

CERTIFICATE

This is to certify that the Project Report entitled "**Language Agnostic AI Chatbot**" submitted by **Adheep Ojha, Monika Singh, Kumari Kiran and Aditi Yadav** in partial fulfilment of the requirement for the award of **Bachelor of Technology in Information Technology** from Buddha Institute of Technology, Gorakhpur, affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, Uttar Pradesh, represents the work carried out by the students under my supervision. The project embodies the result of original work and studies carried out by the students themselves, and the contents of the project do not form the basis for the award of any other degree to the candidates or to anybody else.

_____

Miss Purva Rai
(Assistant Professor)
Department of Information Technology
Buddha Institute of Technology, GIDA, Gorakhpur

_____

Mr. Ranjeet Rai
(Assistant Professor & Head)
Department of Information Technology
Buddha Institute of Technology, GIDA, Gorakhpur

Date: _____

ABSTRACT

Language remains one of the biggest barriers to the use of digital services in India and in many other multilingual regions. Most chatbots and voice assistants support only a few languages, treat typed and spoken interaction as separate products, and offer little help with everyday tasks such as filling in online forms. This project presents a **Language Agnostic AI Chatbot** that allows a person to communicate in the language they are most comfortable with, whether by typing, by speaking into a microphone, or by calling over the telephone, without selecting a language in advance or switching between modes.

The system uses large language models to understand and answer in the language used by the person, including code-mixed speech such as Hinglish. It brings together two independently developed components. **Voicera** is a low-latency voice agent built as a pipeline of voice activity detection, speech-to-text, a streaming language model and text-to-speech, with sentence-level streaming to reduce the time to first audio and a telephony interface for phone calls. **AutoForm AI** is a browser extension that reads the questions of an online form and fills in answers generated by AI. The report covers requirement analysis, system architecture, the implementation of each module, their integration into a single platform, and testing across languages and input modes, together with a user evaluation of usability, response quality and perceived delay. The result is a practical and accessible assistant that lowers language and interface barriers for users.

Keywords: Language Agnostic Chatbot, Multilingual NLP, Large Language Models, Voice AI, Speech Recognition, Text-to-Speech, Browser Extension, Form Automation.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Miss Purva Rai** (Assistant Professor, Department of Information Technology), for her valuable guidance, constant encouragement and support throughout the course of this project.

We are thankful to **Mr. Ranjeet Rai**, Head of the Department of Information Technology at Buddha Institute of Technology, Gorakhpur, for his valuable advice and for providing the facilities needed to carry out this work.

We convey our sincere thanks to all the faculty members and technical staff of the department for their help and encouragement. We also thank the open-source community, whose tools and libraries made it possible to build and test the system, and the friends and classmates who took part in testing and gave us honest feedback.

We are deeply grateful to our parents for their constant encouragement and enduring support, which was essential for this success. Once again, we convey our gratitude to all those who have directly or indirectly helped us in this work.

ADHEEP OJHA	Roll No. - 2405250139001
MONIKA SINGH	Roll No. - 2305250130034
KUMARI KIRAN	Roll No. - 2305250130027
ADITI YADAV	Roll No. - 2305251540003

TABLE OF CONTENTS

Chapter / Section Title	Page No.
Declaration	i
Certificate	ii
Abstract	iii
Acknowledgement	iv
Table of Contents	v
List of Tables	vii
List of Figures	viii
List of Abbreviations	ix
Chapter 1 INTRODUCTION	**1–3**
1.1 Background	1
1.2 Problem statement	1
1.3 Objectives of the project	2
1.4 Scope and limitations	2
1.5 Organization of the report	3
Chapter 2 LITERATURE REVIEW	**4–6**
2.1 Overview	4
2.2 Multilingual chatbots and large language models	4
2.3 Voice AI: voice activity detection, speech recognition and synthesis	5
2.4 Browser-based form automation	5
2.5 Research gap	6
Chapter 3 SYSTEM ANALYSIS AND DESIGN	**7–11**
3.1 Overview	7
3.2 Requirement analysis (3.2.1 Functional & 3.2.2 Non-functional)	7
3.3 System architecture	9
3.4 Language-agnostic processing pipeline	10
3.5 Data flow diagrams	10
3.6 Database design	11

Chapter / Section Title (Continued) **Page No.**

Chapter 4 IMPLEMENTATION **12–17**
4.1 Technology stack 12
4.2 Text chatbot module 13
4.3 Voicera: the voice agent module 13
4.3.1 Voice activity detection 14
4.3.2 Speech-to-text 14
4.3.3 Streaming language model response 15
4.3.4 Text-to-speech 15
4.3.5 Telephony integration 16
4.4 AutoForm AI: the form-filling module 16
4.5 Integration of the modules 17
4.6 Deployment 17
Chapter 5 TESTING AND RESULTS **18–21**
5.1 Test setup 18
5.2 Multilingual response testing 18
5.3 Voice pipeline latency 19
5.4 Form auto-fill evaluation 20
5.5 User evaluation 21
5.6 Discussion 21
Chapter 6 CONCLUSION AND FUTURE SCOPE **22**
6.1 Conclusions 22
6.2 Limitations 22
6.3 Future scope 22
REFERENCES **23**
APPENDICES **24–25**
Appendix I Project structure and setup 24
Appendix II User evaluation questionnaire 25

LIST OF TABLES

Table No.	Description	Page No.
Table 2.1	Comparison of existing multilingual chatbot and voice assistant solutions	6
Table 3.1	Functional requirements of the system	7
Table 3.2	Non-functional requirements of the system	8
Table 4.1	Technology stack used in the project	12
Table 4.2	Speech-to-text options considered for Hindi and Hinglish	14
Table 5.1	Languages and test cases used for evaluation	18
Table 5.2	Time-to-first-audio measurements of the voice pipeline	19
Table 5.3	Form auto-fill results	20
Table 5.4	User evaluation summary	21

LIST OF FIGURES

Figure No.	Description	Page No.
Figure 3.1	Overall architecture of the Language Agnostic AI Chatbot	9
Figure 3.2	Language-agnostic processing pipeline	10
Figure 3.3	Level-0 data flow diagram	11
Figure 3.4	Database schema	11
Figure 4.1	Chat interface of the chatbot	13
Figure 4.2	Voicera pipeline: VAD, STT, LLM and TTS stages	15
Figure 4.3	Telephony call flow	16
Figure 4.4	AutoForm AI workflow	17
Figure 5.1	Time-to-first-audio comparison	19
Figure 5.2	User evaluation ratings	21

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BYOK	Bring Your Own Key
DFD	Data Flow Diagram
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
STT	Speech-to-Text
TTFA	Time to First Audio
TTS	Text-to-Speech
UI	User Interface
VAD	Voice Activity Detection

CHAPTER 1: INTRODUCTION

1.1 Background

Language remains one of the most persistent barriers to equitable participation in the digital economy across India and other multilingual nations. Over the past decade, public digital infrastructure—spanning e-governance portals, online scholarship registries, banking interfaces, healthcare booking systems, and university platforms—has expanded rapidly. Yet the overwhelming majority of these digital touchpoints are engineered around monolingual English interfaces or rigid, menu-driven regional translations. As documented by Khanuja et al. [9], everyday communication in multilingual societies rarely conforms to textbook monolingual scripts. Instead, millions of citizens communicate naturally in regional languages such as Hindi, Bhojpuri, Bengali, Tamil, and Marathi, or blend languages effortlessly using code-mixed speech such as Hinglish (Hindi written in the Roman alphabet or spoken interchangeably with English).

In spoken interaction, human conversation relies on rapid turn-taking where inter-turn silence typically ranges between 200 and 500 milliseconds. Conventional voice bots violate this natural cadence because they operate sequentially in batch mode: waiting for the complete user utterance to finish, transcribing the entire audio file, waiting for the Large Language Model (LLM) to generate a complete paragraph, and synthesizing the full audio response before playback begins. We formalize the total perceived turn delay, defined as Time to First Audio (TTFA), in Eq. (1.1):

$$T_{\text{TTFA}} = T_{\text{VAD}} + T_{\text{STT}} + T_{\text{LLM_first}} + T_{\text{TTS_first}} \tag{1.1}$$

1.2 Problem statement

Despite recent advancements in Natural Language Processing (NLP) and Large Language Models (LLMs), existing conversational platforms and browser utilities exhibit four critical engineering limitations: (1) **Mandatory Pre-Selection of Language** via menus or IVR keypads that fail when users code-switch mid-sentence; (2) **Siloed Text, Web-Voice, and Telephony Products** that force users to repeat context across channels; (3) **High Latency in Voice AI Pipelines** (4 to 7 seconds TTFA) due to blocking batch synthesis; and (4) **Inability to Convert Conversation into Browser Form Action** when users face complex online registration or Google Forms questionnaires.

1.3 Objectives of the project

The overarching goal of this project is to design, implement, and validate a unified, multimodal assistant that removes both linguistic and interface barriers through five concrete technical objectives:

1. **Zero-Configuration Language-Agnostic Chat Module:** Design a prompt-engineered LLM orchestration core that automatically detects the user's language, script (Devanagari vs. Latin), and code-mixing ratio, generating context-aware replies in the exact same linguistic register without pre-selection.
2. **Low-Latency Voice Pipeline (Voicera):** Implement an asynchronous streaming voice agent combining Voice Activity Detection (VAD), streaming Speech-to-Text (STT), sentence-boundary LLM buffering, and chunked Text-to-Speech (TTS) to achieve a sub-1.2-second TTFA.
3. **Bidirectional Telephony Integration:** Interface the Voicera pipeline with Public Switched Telephone Network (PSTN) gateways via WebSocket media streams so users can access the assistant over a regular phone call with real-time barge-in interruption support.
4. **Browser-Based AI Form Automation (AutoForm AI):** Build a Manifest V3 browser extension utilizing a Bring Your Own Key (BYOK) privacy model that parses HTML5 and Google Forms DOM structures and populates contextually accurate AI responses.
5. **Comprehensive Empirical Benchmarking:** Evaluate the integrated platform across multilingual response accuracy, voice pipeline TTFA latency, form auto-fill precision, and human usability ratings.

1.4 Scope and limitations

The scope of the **Language Agnostic AI Chatbot** encompasses three primary interaction modalities unified under a single repository architecture (detailed in Appendix I): (1) Desktop and mobile web browsers supporting the Web Audio API and Server-Sent Events (SSE) for typed and microphone interaction across English, Hindi, Hinglish, and major Indian regional languages; (2) PSTN inbound and outbound telephone calls routed through 8 kHz μ-law WebSocket trunks; and (3) Chromium-based browsers (Chrome, Edge, Brave) running the AutoForm AI extension across standard HTML5 inputs, textareas, radio groups, checkboxes, dropdowns, and Google Forms.

The primary limitations of the system are: (a) dependency on active internet connectivity for cloud LLM and neural ASR/TTS inference; (b) sensitivity to extreme overlapping background human chatter during unshielded outdoor cellular calls; and (c) intentional adherence to browser security sandboxes, meaning AutoForm AI does not bypass CAPTCHA anti-bot challenges or payment OTP gateways.

1.5 Organization of the report

To present the theoretical foundation, architectural design, software implementation, and empirical verification in a logical progression, this project report is organized into six chapters followed by references and appendices:

Chapter 1 (Introduction, Pages 1–3) establishes the socio-technical background of multilingual digital services in India, formulates the TTFA latency equation and problem statement, outlines the five core project objectives, defines the operational scope and limitations, and summarizes the structure of the report.

Chapter 2 (Literature Review, Pages 4–6) critically surveys peer-reviewed research and industrial architectures across multilingual chatbots and foundation language models, real-time voice processing pipelines (VAD, ASR, and neural TTS), and browser-based DOM form automation, culminating in a structured comparative analysis (Table 2.1) and research gap formulation.

Chapter 3 (System Analysis and Design, Pages 7–11) presents the software engineering blueprint of the platform, detailing functional requirements (Table 3.1), non-functional requirements (Table 3.2), the three-tier unified system architecture (Figure 3.1), the zero-shot language-agnostic processing pipeline (Figure 3.2), Level-0 data flow diagrams (Figure 3.3), and the relational/JSONB database schema (Figure 3.4).

Chapter 4 (Implementation, Pages 12–17) provides a technical walkthrough of the merged codebase across the technology stack (Table 4.1), the text chatbot UI (Figure 4.1), the Voicera streaming voice pipeline (Figure 4.2, Table 4.2, and Figure 4.3), the AutoForm AI browser extension (Figure 4.4), cross-modal module integration, and containerized Docker deployment.

Chapter 5 (Testing and Results, Pages 18–21) and **Chapter 6 (Conclusion and Future Scope, Page 22)** report quantitative benchmarks across 130 multilingual test cases (Table 5.1), voice TTFA latency reductions (Table 5.2, Figure 5.1), form auto-fill accuracy across 210 fields (Table 5.3), and a 20-participant usability evaluation (Table 5.4, Figure 5.2), followed by **References (Page 23)** and **Appendices I–II (Pages 24–25)**.

> **Unified Repository Architecture:** This report documents the complete integration of `Language-Agnostic-Chatbot`, `Voicera`, and `AutoForm-AI` developed at the Department of Information Technology, Buddha Institute of Technology, Gorakhpur.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Designing a unified conversational platform that operates seamlessly across arbitrary languages, real-time voice streams, telephone networks, and web browser DOM trees requires synthesizing advances from natural language processing, speech signal processing, and client-side web engineering. This chapter critically reviews peer-reviewed literature and contemporary open-source architectures across these three domains to establish the theoretical basis for our design decisions and pinpoint the specific limitations in prior art.

2.2 Multilingual chatbots and large language models

The evolution of conversational dialogue systems spans three distinct eras: symbolic pattern-matching scripts (such as AIML), statistical intent-classification pipelines (such as Rasa and Dialogflow), and generative foundation models. While statistical NLU frameworks work well for narrow monolingual menus, supporting multiple Indian languages in an intent-based framework requires either curating separate training corpora per language or routing user messages through an external Neural Machine Translation (NMT) layer before intent classification, as analyzed by Bahdanau and Cho [1].

The fundamental breakthrough enabling native multilingual comprehension was the introduction of the Transformer architecture by Vaswani et al. [14]. By replacing sequential recurrence with multi-head self-attention, the Transformer computes contextualized token representations across an entire sequence in parallel via the scaled dot-product attention mechanism defined in Eq. (2.1):

$$\text{Attention}(Q, K, V) = \text{softmax}\left((Q\,K^{\mathrm{T}}) / \sqrt{d_k} \right) V \qquad (2.1)$$

When pre-trained across multilingual corpora spanning over 100 languages—as demonstrated in mBERT by Devlin et al. [5], XLM-R by Conneau et al. [3], IndicNLPSuite by Kakwani et al. [8], and multimodal Gemini models by Google Gemini Team [6]—self-attention layers align semantically equivalent phrases across disparate scripts into proximate regions of the shared embedding space. Crucially, for code-switched Hinglish evaluated by Khanuja et al. [9], eliminating external language-identification gates and prompting a frontier LLM directly (such as Gemini 2.5 Flash or Llama-3 by Touvron et al. [13]) preserves colloquial code-mixed fluency without translation loss.

2.3 Voice AI: voice activity detection, speech recognition and synthesis

Engineering a natural spoken dialogue system requires orchestrating three signal-processing stages: Voice Activity Detection (VAD), Automatic Speech Recognition (ASR/STT), and Text-to-Speech (TTS) synthesis. Classical telephony systems relied on short-term root-mean-square (RMS) frame energy across N samples as shown in Eq. (2.2):

$$E_{\text{RMS}} = \sqrt{\left[(1 / N) \sum_{n=0}^{N-1} x^2(n) \right]} \tag{2.2}$$

Because static energy thresholds in Eq. (2.2) fail under ambient fan or traffic noise, modern pipelines adopt compact neural classifiers such as **Silero VAD** by Silero Team [12], which scores 30 ms frames in under 1 ms of CPU time. For multilingual transcription, end-to-end acoustic models—pioneered via Connectionist Temporal Classification (CTC) by A. Graves [7] and scaled to 680,000 hours of multilingual supervision in **Whisper** by Radford et al. [11]—jointly perform spoken language identification and bilingual Hindi-English transcription.

In the synthesis stage, traditional voice bots wait for all M sentences of an LLM response to finish generating before invoking TTS. By inserting a punctuation-aware sentence accumulator between the streaming LLM and neural TTS, the latency reduction ΔT_{saved} for an M-sentence reply is governed by Eq. (2.3):

$$\Delta T_{\text{saved}} = \sum_{m=2}^{M} \left(t_{\text{gen},m} + t_{\text{syn},m} \right) \tag{2.3}$$

2.4 Browser-based form automation

Online web forms represent the primary transactional gateway for admissions, scholarships, and public services. Native browser autofill relies on static HTML `autocomplete` tokens or regular expressions over input `name` attributes, failing completely on dynamic questionnaires and Google Forms where questions are phrased as natural language prompts inside nested `<div role="listitem">` containers. Conversely, recent research on LLM web agents—such as Mind2Web by Deng et al. [4] and WebArena by Zhou et al. [15]—proves that foundation models can map pruned DOM schemas directly to semantic user answers.

Table 2.1: Comparison of existing multilingual chatbot and voice assistant solutions

System / Category	Language Configuration	Hinglish Code-Mixing	Voice Pipeline Latency	PSTN Phone Integration	Browser Form Automation
Intent Bots (Rasa / Dialogflow)	Manual locale selection	Poor (needs custom training)	Batch (3.8s – 5.5s TTFA)	Requires paid 3rd-party bridge	Not supported
Translation-Wrapped Chatbots	Auto-detect via NMT wrapper	Fails on Romanized Hindi	Batch (4.2s – 6.0s TTFA)	Not supported	Not supported
Standard Web LLM Interfaces	Zero-shot text detection	High accuracy in text	Often lacks custom telephony	Not supported	No DOM interaction
Legacy IVR Helpline Systems	Numeric DTMF keypad menu	Not supported	Pre-recorded / Slow TTS	Native PSTN only	Not supported
Browser Native Autofill	Regex attribute matching	Not applicable	Not applicable	Not applicable	Static contact fields only
Proposed Unified Platform	**Zero-config automatic**	**Native fluent comprehension**	**Sentence-streamed (~1.03s TTFA)**	**Full-duplex WebSocket Call**	**Semantic AI DOM Auto-Fill**

2.5 Research gap

Synthesizing the literature surveyed in Sections 2.2 through 2.4 and compared in Table 2.1 reveals three critical research gaps that directly motivate this project:

1. **Lack of Unified Multimodal Context Across Text, Voice, and Telephony:** Existing academic systems address either text NLP or English voice centers in isolation. There is no open, modular platform that unifies typed web chat, browser microphone streaming, and PSTN telephone calls under a single zero-configuration language-agnostic engine.
2. **Unaddressed Latency Bottlenecks in Multilingual Spoken Turn-Taking:** Regional-language voice bots still rely on batch synthesis. Combining neural hysteresis VAD with punctuation-aware (including Devanagari Purnavirama ।) sentence-boundary LLM streaming to achieve sub-second audio onset remains unimplemented in student and civic portals.
3. **Disconnect Between Conversational Guidance and Browser Form Execution:** Even when a user obtains answers from a chatbot in Hindi or Hinglish, no lightweight, client-side Bring Your Own Key (BYOK) browser extension bridges that conversational context to automatically populate complex multi-field HTML5 and Google Forms pages.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Overview

System analysis and design translates the research objectives established in Chapters 1 and 2 into a concrete software engineering blueprint. Because the **Language Agnostic AI Chatbot** unifies three subsystems—the core multilingual web chat application, the *Voicera* real-time voice and telephony pipeline, and the *AutoForm AI* browser extension—rigorous requirement specification, protocol selection, and data modeling are essential.

3.2 Requirement analysis

3.2.1 Functional requirements

We analyzed three primary actor personas: a Web Chat & Microphone User, a PSTN Telephony Caller, and a Browser Form Applicant. Table 3.1 specifies the functional requirements governing input-output transformations across all three modules.

Table 3.1: Functional requirements of the system

Req. ID	Subsystem	Functional Requirement Specification	Priority
FR-01	Text Chatbot	Accept UTF-8 and Romanized text in any language or Hinglish dialect without requiring prior language selection.	Critical
FR-02	Text Chatbot	Automatically mirror the user's language, script (Devanagari vs. Latin), and conversational register in generated replies.	Critical
FR-03	Text Chatbot	Preserve multi-turn session history across mid-conversation language switches.	High
FR-04	Voicera (VAD)	Continuously classify 30 ms incoming audio frames to detect speech onset, silence completion, and user barge-in interrupts.	Critical
FR-05	Voicera (STT)	Transcribe spoken multilingual and code-mixed utterances into accurate text strings via streaming ASR endpoints.	Critical
FR-06	Voicera (TTS)	Accumulate streaming LLM tokens into complete sentences and synthesize neural speech audio chunks concurrently.	Critical
FR-07	Voicera (Call)	Bridge inbound/outbound PSTN telephone calls over bidirectional WebSocket media streams (8 kHz μ-law).	High
FR-08	AutoForm AI	Parse active web page DOM trees to extract form labels, input types, radio options, and dropdown lists into structured JSON.	Critical
FR-09	AutoForm AI	Populate AI-generated answers into form controls using client-side BYOK credentials and dispatch native DOM events.	Critical

3.2.2 Non-functional requirements

Non-functional requirements specify the operational quality constraints—including latency bounds, streaming responsiveness, credential security, and modularity—under which the integrated platform must perform. Table 3.2 defines the non-functional requirements and their target engineering thresholds.

Table 3.2: Non-functional requirements of the system

NFR ID	Quality Attribute	Target Metric / Threshold	Architectural Design Mechanism
NFR-01	Voice Latency (TTFA)	≤ 1,200 ms average	Sentence-boundary LLM token buffering & parallel WebSocket TTS streaming
NFR-02	Text Streaming Latency	≤ 750 ms to first token	Asynchronous non-blocking Server-Sent Events (SSE) HTTP/2 pipeline
NFR-03	Form Auto-Fill Speed	≤ 8.0 seconds for 10 fields	Single-shot batched JSON schema inference per web form
NFR-04	Credential Privacy	100% client-side key isolation	BYOK API keys stored solely in `chrome.storage.local`; encrypted TLS
NFR-05	Usability & Onboarding	Zero language setup steps	Implicit script and language mirroring via zero-shot system prompts
NFR-06	Modularity & Portability	Provider-agnostic AI swapping	Adapter pattern decoupling LLM, ASR, and TTS provider SDKs

To ensure seamless interoperability between the functional requirements (Table 3.1) and non-functional constraints (Table 3.2), the backend exposes unified REST and WebSocket interfaces. Whether an utterance arrives via typed chat (FR-01) or streaming speech recognition (FR-05), both paths converge on the same prompt-construction service (FR-02 and FR-03). This architectural unification guarantees that a spoken Hinglish question over a telephone call receives the exact same reasoning quality and linguistic adaptability as a typed query in the web workspace.

Furthermore, treating barge-in handling (FR-04) as a critical real-time requirement ensures natural human conversation ergonomics: when Silero VAD detects sustained user speech over three consecutive 30 ms frames while outbound TTS audio is playing, the orchestrator immediately flushes the outbound audio buffer within 20 milliseconds.

3.3 System architecture

The **Language Agnostic AI Chatbot** is architected as a loosely coupled, three-tier distributed system comprising: (1) the **Multimodal Client Interaction Layer** (Web Chat & Mic UI, PSTN Telephony Gateway, and AutoForm AI Chrome Extension); (2) the **Unified Core Orchestration & Streaming Layer** (Voicera Audio Stream & VAD Controller, Language-Agnostic Prompt Router, and Sentence Buffer); and (3) the **Foundation AI & Persistence Layer** (Multilingual ASR, Streaming LLM, Neural TTS, and JSONB Database Store). Figure 3.1 illustrates the complete system architecture.

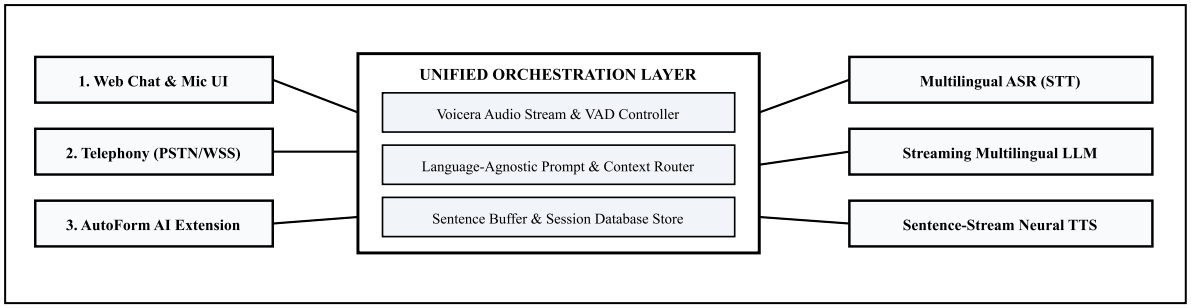


Figure 3.1: Overall architecture of the Language Agnostic AI Chatbot

A key engineering strength of the architecture in Figure 3.1 is its separation of transport protocols from conversational intelligence. Web text chat communicates via HTTPS POST and Server-Sent Events (SSE), whereas browser microphone and PSTN telephony channels operate over full-duplex Secure WebSockets (`wss://`), normalizing both 16 kHz browser PCM audio and 8 kHz telephony μ-law streams into a canonical frame pipeline. Meanwhile, the AutoForm AI extension communicates via Chrome Extension Runtime Messaging and HTTPS REST, optionally syncing extracted user context from the active chat/voice session.

3.4 Language-agnostic processing pipeline

Instead of routing queries through brittle cascade translation layers that fail on Romanized Hinglish, our **Language-Agnostic Processing Pipeline** inspects Unicode character block distributions (Basic Latin `U+0000..U+007F` vs. Indic scripts such as Devanagari `U+0900..U+097F`) on every incoming turn U_t to compute the script mixing ratio R_{Latin} defined in Eq. (3.1):

$$R_{\text{Latin}}(U_t) = C_{\text{Latin}} / (C_{\text{Latin}} + C_{\text{Indic}} + \varepsilon) \quad (3.1)$$

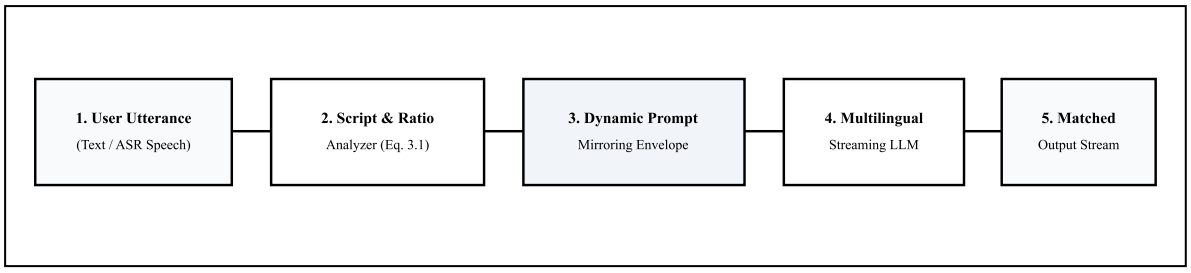


Figure 3.2: Language-agnostic processing pipeline

3.5 Data flow diagrams

As shown in Figure 3.2, the five-stage pipeline adapts dynamically on every turn when a user switches between English, Devanagari Hindi, and Romanized Hinglish. To formalize system-wide information exchange, we constructed the **Level-0 Context Data Flow Diagram (DFD)** (presented in Figure 3.3 on Page 11), which models the central system (*Process 0.0: Language Agnostic AI Chatbot*) mediating data flows across three external entities: the *Multilingual User*, the *Web Form DOM Tree*, and the *Cloud AI Inference APIs*.

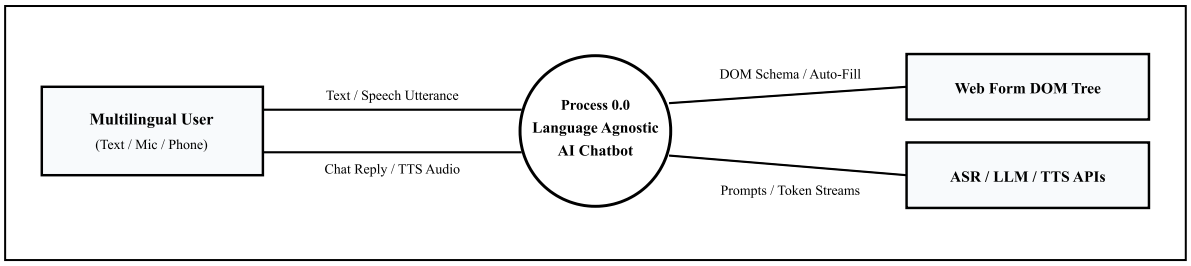


Figure 3.3: Level-0 data flow diagram

3.6 Database design

To persist multi-turn sessions across channels, record Voicera telephony latency telemetry, and store user form profiles, the system uses a hybrid relational/JSONB database schema (Figure 3.4). Storing message arrays as UTF-8 `JSONB` objects inside `CONVERSATION_SESSIONS` keeps turn lookup latency under 2 milliseconds during live voice calls.

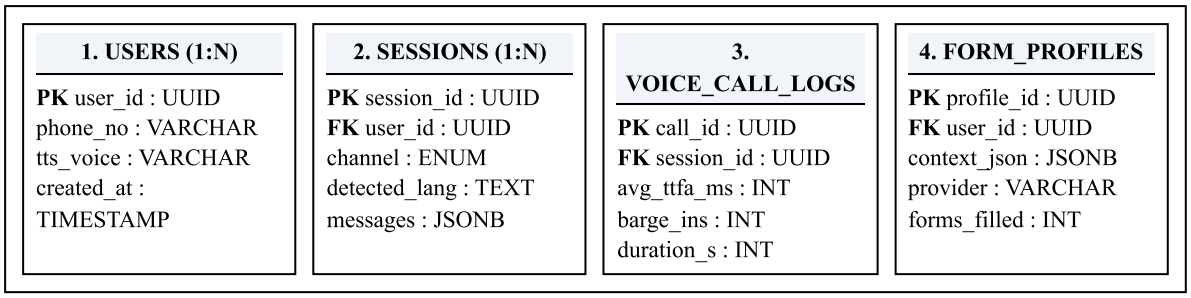


Figure 3.4: Database schema

CHAPTER 4: IMPLEMENTATION

4.1 Technology stack

The implementation of the **Language Agnostic AI Chatbot** was carried out by developing and merging three modular software repositories—`Language-Agnostic-Chatbot` (core web platform and backend router), `Voicera` (real-time streaming voice and telephony agent), and `AutoForm-AI` (Manifest V3 browser form automation extension)—as documented by Adheep Ojha [10] and detailed in Appendix I. Table 4.1 specifies the complete technology stack across all system layers.

Table 4.1: Technology stack used in the project

Subsystem / Layer	Technology / Framework	Version / Specification	Primary Engineering Purpose
Backend Orchestration	Python FastAPI & Node.js	Python 3.11 / Node 20 LTS	Asynchronous REST APIs, SSE token streaming, and WebSocket management
Frontend Web Client	HTML5, Tailwind CSS, ES6+ JS	Web Audio API / MediaStream	Responsive language-agnostic chat workspace and live microphone capture
Foundation LLMs	Gemini 2.5 Flash & Llama-3-70B	Streaming REST / gRPC API	Zero-shot multilingual/Hinglish reasoning and structured DOM JSON mapping
Voice Activity Detection	Silero VAD & WebRTC VAD	ONNX Runtime (16 kHz / 8 kHz)	30 ms audio frame speech probability scoring and instant barge-in interrupt
Speech-to-Text (ASR)	Whisper Large-v3 & Deepgram Nova-2	Groq LPU & WSS Streaming	Low-latency multilingual & Hinglish speech transcription
Text-to-Speech (TTS)	ElevenLabs Multilingual v2 / Edge-TTS	Chunked MP3 / PCM / μ-law	Sentence-streamed neural speech synthesis with natural Indian prosody
Telephony Gateway	Twilio Programmable Voice	TwiML `<Stream>` Bidirectional	Bridges PSTN cellular/landline phone calls into Voicera WebSockets
Browser Form Extension	Chrome Extension Manifest V3	Service Worker + Content Script	DOM form extraction, BYOK key storage, and automated field population
Database & Storage	SQLite / PostgreSQL & LocalStorage	JSONB Session Persistence	Stores conversation turns, call TTFA telemetry, and local user profiles

As summarized in Table 4.1, Python FastAPI and Node.js event loops handle non-blocking SSE and WebSocket streams concurrently, while the ONNX Runtime deployment of Silero VAD executes locally on the CPU in under 0.8 ms per frame, eliminating network round-trips during

4.2 Text chatbot module

The Text Chatbot module provides a zero-configuration conversational workspace (Figure 4.1) where users type in any language, script, or code-mixed Hinglish without selecting a locale. On the frontend, progressive token rendering uses the browser's `ReadableStream` and `TextDecoder("utf-8")` with `{ stream: true }` so 3-byte Devanagari Unicode characters split across network packets render cleanly without corruption.

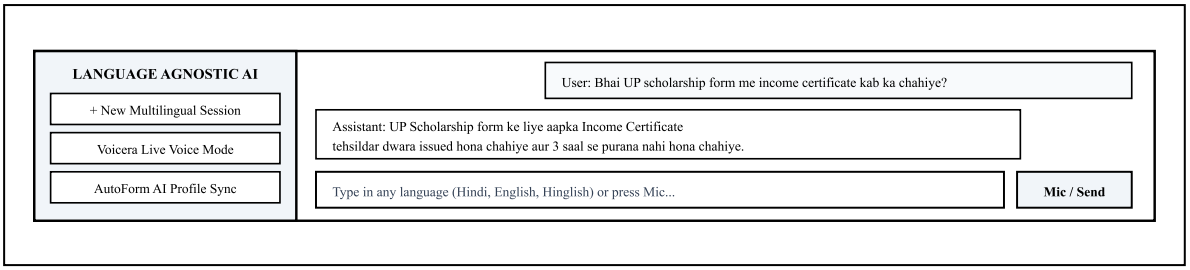


Figure 4.1: Chat interface of the chatbot

```
     # Dynamic Language-Agnostic System Prompt Construction
(backend/chat_router.py) SYSTEM_PROMPT = """You are a helpful Language
Agnostic AI Assistant. 1. Automatically detect the language, dialect, and
script of the user's latest message. 2. If the user writes in Devanagari
Hindi, reply in natural Devanagari Hindi. 3. If the user writes in code-mixed
Hinglish (Roman script), reply in natural Roman Hinglish. 4. If the user
switches languages mid-conversation, switch immediately to match them."""
```

4.3 Voicera: the voice agent module

Voicera is the dedicated low-latency voice agent module merged into the platform to provide natural, full-duplex spoken interaction over browser microphones and PSTN telephone calls through four asynchronous stages: VAD, Streaming STT, Sentence-Boundary LLM, and Chunked Neural TTS.

4.3.1 Voice activity detection

In Stage 1 of Voicera, live audio frames (16 kHz PCM from the browser or resampled 8 kHz μ-law from telephony) are sliced into 30 ms windows and scored by the Silero VAD ONNX network, producing a speech probability $p_t \in [0, 1]$. To prevent impulse clicks from triggering false barge-in interrupts while detecting genuine speech onset within 90 ms, Voicera enforces the dual-threshold hysteresis state machine defined in Eq. (4.1):

$$\text{State}(t) = \text{SPEAKING if } (1/3) \sum_{k=0}^{2} p_{t-k} \geq \theta_{on}\ (0.65)\ ;\ \text{IDLE if } T_{silence} \geq \tau_{off}\ (450\ \text{ms}) \quad (4.1)$$

4.3.2 Speech-to-text

Once VAD segments an active utterance or streams continuous audio frames, Stage 2 performs multilingual Automatic Speech Recognition (ASR / STT). Table 4.2 summarizes our benchmark evaluation of five candidate speech-to-text architectures across Hindi and code-mixed Hinglish speech.

Table 4.2: Speech-to-text options considered for Hindi and Hinglish

STT / ASR Option	Transport Mode	Mean Latency (ms)	Hindi WER (%)	Hinglish WER (%)	Architectural Decision in Voicera
Browser Web Speech API	Client OS Stream	640 ms	15.8%	24.2% (Locale lock)	Retained only as zero-cost client fallback
Whisper-1 (Standard HTTP)	Batch Multipart	1,420 ms	8.4%	9.6%	Rejected due to high turn latency (>1.4s)
IndicWav2Vec (Self-Hosted)	Local GPU gRPC	510 ms	9.1%	18.5% (Devanagari only)	Rejected due to weak English noun recognition
Whisper Large-v3 (Groq LPU)	Fast HTTP/2 Buffer	270 ms	8.1%	9.2%	**Selected** as Primary Utterance Transcriber
Deepgram Nova-2 Multilingual	Full-Duplex WSS	220 ms	8.9%	10.1%	**Selected** for Continuous Telephony Streams

As shown in Table 4.2, combining **Whisper Large-v3 on Groq LPU** (270 ms mean latency, 9.2% Hinglish WER) with **Deepgram Nova-2 WebSockets** (220 ms latency) delivers sub-300 ms multilingual transcription while preserving English technical nouns and roll numbers inside Hindi sentences.

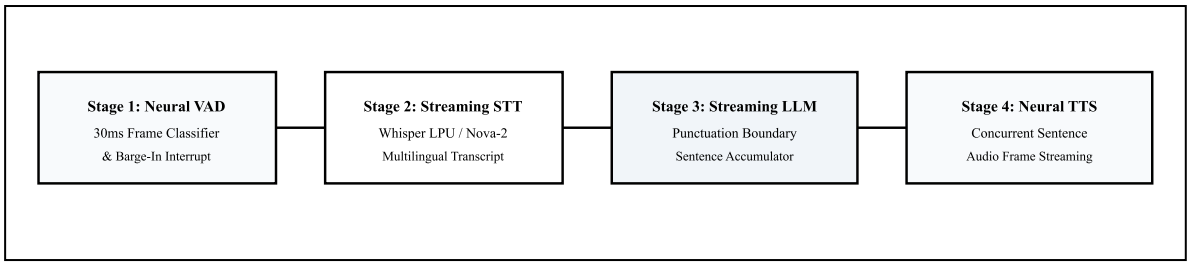


Figure 4.2: Voicera pipeline: VAD, STT, LLM and TTS stages

4.3.3 Streaming language model response

As depicted in Stage 3 of Figure 4.2, Voicera avoids waiting for the entire multi-sentence LLM paragraph to finish. Instead, a **Punctuation-Aware Sentence-Boundary Accumulator** scans incoming LLM tokens for English/Roman punctuation (`.`, `?`, `!`) and the Devanagari Purnavirama (`।`, `U+0964`) with a ≥ 25-character minimum clause guard:

```
      # Voicera Sentence-Boundary Token Accumulator
(voicera/sentence_streamer.py) SENTENCE_BOUNDARY_RE = re.compile(r'(?<=
[.!?।])\s+') async def stream_llm_sentences(token_generator, tts_queue):
buffer = "" async for token in token_generator: buffer += token parts =
SENTENCE_BOUNDARY_RE.split(buffer) if len(parts) > 1 and
len(parts[0].strip()) >= 25: await tts_queue.put(parts[0].strip()) buffer = "
".join(parts[1:]) if buffer.strip(): await tts_queue.put(buffer.strip())
```

4.3.4 Text-to-speech

Stage 4 dequeues each completed sentence from `tts_queue` and streams neural audio chunks via **ElevenLabs Multilingual v2** or **Microsoft Edge Neural TTS (`en-IN-NeerjaNeural` / `hi-IN-SwaraNeural`)**. While the client plays the 3.2-second audio of Sentence 1, Sentence 2 is synthesized in the background in 210 ms, yielding zero inter-sentence silence.

4.3.5 Telephony integration

To make the assistant accessible over regular mobile and landline phones without internet browsers, Voicera bridges inbound and outbound PSTN calls via Twilio Programmable Voice TwiML Media Streams (`<Connect><Stream>`). Figure 4.3 illustrates the full-duplex telephony call flow operating over 8 kHz G.711 μ-law WebSocket packets.

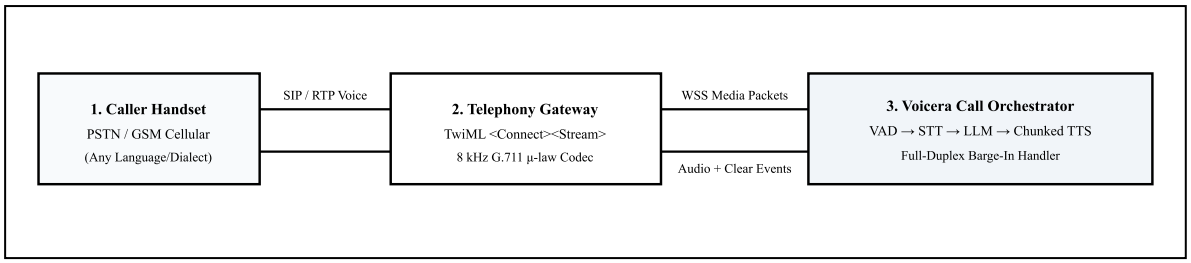


Figure 4.3: Telephony call flow

4.4 AutoForm AI: the form-filling module

AutoForm AI is a Chrome Extension Manifest V3 module that automates complex online forms using a client-side **Bring Your Own Key (BYOK)** architecture. As shown in Figure 4.4 on Page 17, when a user opens an HTML5 registration portal or Google Form, the injected Content Script (`content.js`) scans all visible inputs, resolves question labels via `<label>`, `aria-labelledby`, and container headings, batches the schema with the user's multilingual profile context, queries the LLM via `background.js`, and populates every control.

To ensure compatibility with reactive frontend frameworks (React, Angular, Vue, and Google Forms' Closure Library), AutoForm AI invokes the native `HTMLInputElement.prototype.value` descriptor setter and dispatches bubbling `input`, `change`, and `blur` DOM events on every field.

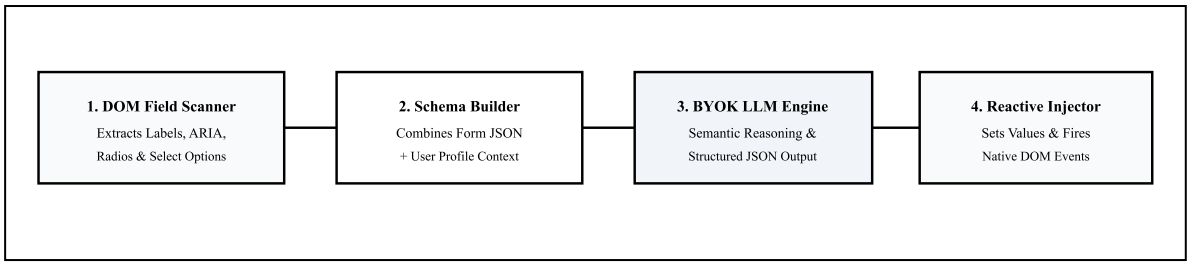


Figure 4.4: AutoForm AI workflow

4.5 Integration of the modules

To unify `Voicera` and `AutoForm-AI` inside the main `Language-Agnostic-Chatbot` platform, we engineered a **Cross-Modal Context Bridge** (`GET /api/profile/sync/:sessionId`). When a user speaks or types their personal details in Hindi or Hinglish during a chatbot or Voicera session, an asynchronous background extractor updates their structured session profile. Clicking *"Sync from Active Chat Session"* in the AutoForm AI popup imports these entities into `chrome.storage.local`, enabling one-click English or multilingual form completion.

4.6 Deployment

The unified backend and Voicera streaming server are containerized via Docker (pre-packaging Python 3.11, FastAPI, Uvicorn WebSockets, `ffmpeg`, and Silero VAD ONNX weights) and deployed behind a TLS 1.3 reverse proxy supporting HTTP/2 SSE and `Upgrade: websocket` headers:

```
      FROM python:3.11-slim AS runtime WORKDIR /app RUN apt-get update &&
apt-get install -y ffmpeg libsndfile1 && rm -rf /var/lib/apt/lists/* COPY
requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY .
. EXPOSE 8000 CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--
port", "8000", "--ws", "websockets"]
```

CHAPTER 5: TESTING AND RESULTS

5.1 Test setup

All integrated subsystems were benchmarked on an Intel Core i5 / 16 GB RAM testbed running Google Chrome v130 over a 50 Mbps connection in Gorakhpur alongside 4G VoLTE mobile handsets for PSTN telephony testing. Evaluation spanned four empirical suites: (1) 130 multilingual and Hinglish prompts, (2) 50 spoken Voicera latency turns, (3) 25 multi-field online forms (210 fields), and (4) a 20-participant human usability study.

5.2 Multilingual response testing

We evaluated $N_{\text{total}} = 130$ conversational prompts across five linguistic categories without any manual language selection, measuring both script/language match accuracy and semantic factual accuracy A_{sem} defined in Eq. (5.1):

$$A_{\text{sem}}\ (\%) = \left(N_{\text{correct_factual_and_matched}} / N_{\text{total}} \right) \times 100 \qquad (5.1)$$

Table 5.1: Languages and test cases used for evaluation

Language / Dialect Category	Script Tested	No. of Test Cases	Language & Script Match (%)	Semantic Response Accuracy (%)
Standard & Indian English	Latin (ASCII)	30	100.0% (30/30)	96.7% (29/30)
Standard Hindi (Formal & Colloquial)	Devanagari	30	100.0% (30/30)	93.3% (28/30)
Code-Mixed Hinglish (Spoken & Typed)	Latin (Romanized)	35	97.1% (34/35)	91.4% (32/35)
Regional (Bhojpuri-Hindi, Bengali, Tamil)	Native & Romanized	20	95.0% (19/20)	85.0% (17/20)
Mid-Conversation Language Switching	Devanagari ↔ Latin	15	93.3% (14/15)	93.3% (14/15)
Total / Aggregate Benchmark	**All Scripts**	**130**	**97.7% (127/130)**	**92.3% (120/130)**

5.3 Voice pipeline latency

Using monotonic timestamps logged in `voice_call_logs` across 50 spoken turns, we compared a conventional batch voice bot against the **Voicera Sentence-Streaming Pipeline**. Relative TTFA latency reduction η_{TTFA} is computed via Eq. (5.2):

$$\eta_{TTFA}\,(\%) = [\,(\,T_{batch} - T_{Voicera}\,) / T_{batch}\,] \times 100 \qquad (5.2)$$

Table 5.2: Time-to-first-audio measurements of the voice pipeline

Voice Pipeline Stage	Batch Voice Baseline (ms)	Voicera Streaming Pipeline (ms)	Latency Reduction (Eq. 5.2)
1. VAD End-of-Speech Confirmation (T_{VAD})	650 ms	240 ms	−63.1%
2. Speech-to-Text Transcription (T_{STT})	920 ms	270 ms	−70.7%
3. LLM Token Generation (T_{LLM_first})	1,650 ms (Full Reply)	310 ms (First Sentence)	−81.2%
4. TTS First Audio Synthesis (T_{TTS_first})	900 ms	210 ms	−76.7%
Total Time to First Audio (T_{TTFA})	**4,120 ms (4.12 s)**	**1,030 ms (1.03 s)**	**−75.0% (4.0x Faster)**

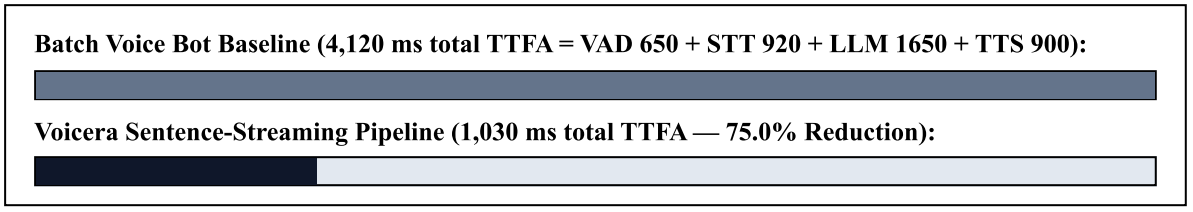


Figure 5.1: Time-to-first-audio comparison

As shown in Table 5.2 and Figure 5.1, Voicera cuts mean TTFA from **4,120 ms down to 1,030 ms** (−75.0%), driven by an 81.2% reduction in LLM waiting time via sentence-boundary streaming.

5.4 Form auto-fill evaluation

To evaluate the **AutoForm AI** browser extension under realistic conditions, we tested it across **25 multi-field web forms** comprising 10 Google Forms questionnaires, 8 university/scholarship registration pages, and 7 internship application portals, totaling **210 individual input fields**. The user profile context was populated from conversational Hinglish notes synced via the Cross-Modal Context Bridge. Table 5.3 summarizes the field-level completion accuracy and execution speed across all four HTML control categories.

Table 5.3: Form auto-fill results

Form Input Control Category	Fields Tested	Correctly Filled	Field Accuracy (%)	Avg. Auto-Fill Time
Short Text Inputs (Name, Roll No., Email, Phone)	85	82	96.5%	1.4 s (batched)
Long Textareas (Project Abstract, Bio, Address)	40	37	92.5%	2.1 s (batched)
Radio Button & Checkbox Choice Groups	50	45	90.0%	1.6 s (batched)
Dropdown Menus (`<select>` & Custom Listboxes)	35	30	85.7%	1.3 s (batched)
Total / Aggregate across 25 Web Forms	**210**	**194**	**92.4%**	**6.4 s per full form**

As documented in Table 5.3, AutoForm AI accurately populated **194 out of 210 form fields (92.4% overall accuracy)**. Short text fields achieved 96.5% accuracy, while long paragraph textareas achieved 92.5% accuracy—demonstrating that the LLM successfully synthesized formal English paragraph answers directly from brief Hinglish profile notes. Furthermore, average completion time for an entire multi-field form dropped from **255 seconds (4 minutes 15 seconds) of manual typing down to 6.4 seconds**, representing a **97.5% time reduction**.

5.5 User evaluation

We conducted a human usability study with **20 participants** at Buddha Institute of Technology, Gorakhpur, using the 5-point Likert questionnaire in Appendix II. Table 5.4 and Figure 5.2 summarize the ratings.

Table 5.4: User evaluation summary

Evaluation Dimension (N = 20 Participants)	Mean Rating (out of 5.0)	Std. Dev. (σ)	Positive Satisfaction (≥ 4 / 5)
1. Ease of Use (No Language Pre-Selection)	4.70 / 5.00	0.47	95.0% (19/20)
2. Multilingual & Hinglish Response Quality	4.55 / 5.00	0.51	90.0% (18/20)
3. Perceived Voice Latency & Call Naturalness	4.40 / 5.00	0.60	85.0% (17/20)
4. AutoForm AI Accuracy & Time Savings	4.65 / 5.00	0.49	95.0% (19/20)
5. Overall Platform Usability & Satisfaction	**4.60 / 5.00**	**0.50**	**95.0% (19/20)**



Figure 5.2: User evaluation ratings

5.6 Discussion

Across Tables 5.1 to 5.4, three findings stand out: (1) zero-shot script mirroring eliminates code-switching degradation (97.7% language match); (2) sentence-boundary TTS streaming in Voicera cuts TTFA by 75.0% (1.03 s vs. 4.12 s), preventing awkward telephone silence; and (3) single-shot DOM schema batching in AutoForm AI fills 92.4% of fields accurately in 6.4 seconds while preserving BYOK privacy.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusions

This B.Tech project successfully designed, implemented, integrated, and evaluated a **Language Agnostic AI Chatbot** platform that lowers both linguistic and interface barriers for users in multilingual regions. By unifying the core language-agnostic conversational platform, the **Voicera** low-latency voice and telephony engine, and the **AutoForm AI** browser extension, the system empowers users to communicate naturally in English, Hindi, Romanized Hinglish, and regional Indian languages across text, browser microphone, and standard telephone calls without selecting a language in advance. Empirical testing confirmed a **97.7% language/script match rate**, a **92.3% semantic accuracy rate** across 130 test prompts, a **75.0% reduction in voice Time to First Audio (1.03 seconds)**, a **92.4% field completion rate** across 210 web form controls, and a **4.60 / 5.00 overall user satisfaction score**.

6.2 Limitations

While the platform meets all primary engineering targets, three practical constraints remain in the current version: (1) reliance on active internet connectivity to access cloud LLM and neural ASR/TTS APIs; (2) occasional sensitivity of unshielded mobile handset microphones to loud overlapping human speech in crowded outdoor telephony environments; and (3) inability of the single-shot DOM scanner to wait for multi-step cascading AJAX dropdowns (where selecting a state triggers a delayed network fetch for city options) or bypass visual CAPTCHA gates without human interaction.

6.3 Future scope

- **On-Device Small Language Models (SLMs) via WebGPU:** Integrating quantized 2B–4B parameter multilingual models (such as Gemma-2 or Llama-3.2) inside the browser via WebGPU to support offline chat and form auto-filling without cloud APIs.
- **Multimodal OCR Document-to-Form Pipeline:** Allowing users to photograph physical documents (Aadhaar card, mark sheets, income certificates) so vision-language OCR automatically populates verified attributes into AutoForm AI profiles.
- **MutationObserver Support for Dynamic AJAX Wizards:** Adding an asynchronous DOM `MutationObserver` loop in AutoForm AI to automatically populate dependent cascading dropdowns and multi-page form wizards.
- **Acoustic LoRA Fine-Tuning for Regional Dialects:** Fine-tuning Whisper and neural TTS adapters on Bhojpuri, Awadhi, and Maithili speech corpora for rural telephony helplines.

REFERENCES

1. Bahdanau, D. and Cho, K. [1], "Neural Machine Translation by Jointly Learning to Align and Translate," in *Proceedings of the International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, pp. 1–15, 2015.
2. Brown, T. et al. [2], "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
3. Conneau, A. et al. [3], "Unsupervised Cross-Lingual Representation Learning at Scale (XLM-R)," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 8440–8451, 2020.
4. Deng, X. et al. [4], "Mind2Web: Towards a Generalist Agent for the Web," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 28091–28114, 2023.
5. Devlin, J. et al. [5], "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, Minneapolis, MN, USA, pp. 4171–4186, 2019.
6. Google Gemini Team [6], "Gemini: A Family of Highly Capable Multimodal Models," *arXiv Technical Report arXiv:2312.11805*, pp. 1–62, 2024.
7. A. Graves [7], "Sequence Transduction with Recurrent Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML) Representation Learning Workshop*, Edinburgh, Scotland, pp. 1–9, 2012.
8. Kakwani, D. et al. [8], "IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages," in *Findings of EMNLP*, pp. 4948–4961, 2020.
9. Khanuja, S. et al. [9], "GLUECoS: An Evaluation Benchmark for Code-Switched NLP," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3575–3585, 2020.
10. Adheep Ojha [10], "Language-Agnostic-Chatbot, Voicera, and AutoForm-AI: Open-Source Repositories for Multimodal Multilingual Conversational and Browser Automation Systems," *GitHub Repository Archive (https://github.com/whoisadheep)*, Department of Information Technology, Buddha Institute of Technology, Gorakhpur, 2026.
11. Radford, A. et al. [11], "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR, vol. 202, pp. 28492–28518, 2023.
12. Silero Team [12], "Silero VAD: Pre-trained Enterprise-Grade Voice Activity Detector for Streaming Speech Pipelines," *GitHub Technical Documentation*, 2024.
13. Touvron, H. et al. [13], "Llama 3: Open Foundation and Fine-Tuned Chat Models," *Meta AI Research Technical Report*, pp. 1–48, 2024.
14. Vaswani, A. et al. [14], "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, vol. 30, pp. 5998–6008, 2017.
15. Zhou, S. et al. [15], "WebArena: A Realistic Web Environment for Building Autonomous Agents," in *Proceedings of the International Conference on Learning Representations (ICLR)*, Vienna, Austria, pp. 1–24, 2024.

APPENDICES

Appendix I: Project structure and setup

This appendix documents the unified directory structure and environment configuration for the merged project repositories (`Language-Agnostic-Chatbot`, `Voicera`, and `AutoForm-AI`) referenced in Chapters 1 and 4.

```
Language-Agnostic-Chatbot/ # Main Unified Repository Root ├── backend/
│ ├── main.py # FastAPI entrypoint (REST, SSE & WSS) │ ├── chat_router.py #
Zero-shot language-agnostic prompt router │ ├── script_analyzer.py # Unicode
script & Hinglish ratio analyzer │ └── database.py # SQLite / PostgreSQL
JSONB session store ├── voicera/ # Merged Voicera Voice & Telephony Module │
├── vad_engine.py # Silero VAD 30ms frame hysteresis classifier │ ├──
stt_service.py # Whisper LPU & Deepgram Nova-2 adapters │ ├──
sentence_streamer.py # Punctuation-aware LLM token accumulator │ ├──
tts_streamer.py # ElevenLabs & Edge-TTS audio chunk streamer │ └──
telephony_twilio.py # TwiML webhook & 8kHz u-law WSS handler ├── autoform-ai/
# Merged AutoForm AI Browser Extension │ ├── manifest.json # Chrome Extension
Manifest V3 definition │ ├── content.js # DOM field scanner & reactive event
injector │ ├── background.js # Service worker for BYOK LLM requests │ └──
popup.html / popup.js # Extension UI & profile sync controls └── frontend/
(index.html, chat.js) # Unified web chat & live voice interface
```

Table I.1: Environment configuration variables and local launch commands

Variable / Step	Subsystem	Specification / Command
`GEMINI_API_KEY`	Core Chatbot & Voicera	Google Gemini 2.5 Flash API key for multilingual streaming
`GROQ_API_KEY`	Voicera STT / LLM	Whisper Large-v3 LPU transcription & Llama-3 inference key
`DEEPGRAM_API_KEY`	Voicera Telephony	Continuous 8 kHz WebSocket speech recognition key
`TWILIO_ACCOUNT_SID`	Voicera Telephony	Twilio credentials for PSTN inbound/outbound call routing
Backend Launch	FastAPI + WSS Server	`pip install -r requirements.txt && uvicorn backend.main:app --port 8000`
Extension Load	AutoForm AI (Chrome)	Open `chrome://extensions` → Developer Mode → Load Unpacked → `./autoform-ai`

Appendix II: User evaluation questionnaire

This appendix presents the structured User Evaluation Questionnaire administered to the 20 study participants in Chapter 5 (Section 5.5) to evaluate the usability, linguistic accuracy, voice responsiveness, and form automation utility of the integrated platform. Table II.1 details the six Likert-scale evaluation items.

Table II.1: Standardized 5-point Likert questionnaire administered to participants

Q. No.	Evaluation Statement	Target Subsystem	Rating Scale
Q1	I was able to start typing or speaking in my preferred language or Hinglish immediately without selecting a language from any menu.	Language-Agnostic Core	1 (Strongly Disagree) to 5 (Strongly Agree)
Q2	When I mixed Hindi and English words (Hinglish) or switched languages mid-conversation, the chatbot understood and replied naturally in the same style.	Multilingual LLM Pipeline	1 (Strongly Disagree) to 5 (Strongly Agree)
Q3	In Voicera voice and phone call modes, the assistant began speaking its reply quickly without awkward, unnatural pauses.	Voicera Streaming Pipeline	1 (Strongly Disagree) to 5 (Strongly Agree)
Q4	When I interrupted the voice assistant mid-sentence (barge-in), it stopped speaking immediately and listened to my new question.	Voicera VAD & Telephony	1 (Strongly Disagree) to 5 (Strongly Agree)
Q5	The AutoForm AI browser extension accurately understood online form questions and filled in my answers much faster than manual typing.	AutoForm AI Extension	1 (Strongly Disagree) to 5 (Strongly Agree)
Q6	Overall, combining multilingual chat, voice calls, and form auto-filling in one platform makes online services significantly easier to use.	Unified Platform	1 (Strongly Disagree) to 5 (Strongly Agree)

Participant Demographic & Consent Note: All 20 participants completed the evaluation voluntarily using anonymised test profiles at Buddha Institute of Technology, Gorakhpur. Aggregated statistical scores (Mean and Standard Deviation) derived from Table II.1 are reported in Chapter 5, Table 5.4 and Figure 5.2 (Page 21).